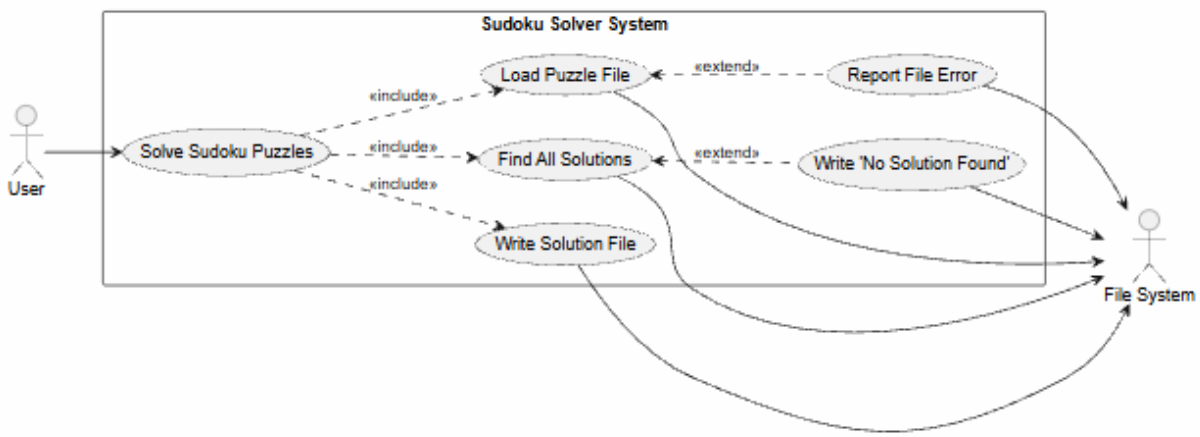
Part 1: Modify the previous user's UML use case diagram:

Used Gemini by Google; made using PlantUML.com code



a. Identified Use Case

Primary Use Case: Solve Sudoku Puzzles

Included Use Cases:

-Load Puzzle File

-Find All Solutions

-Write Solution File

Extended Use Cases:

Report File Error

Write "No Solution Found"

b. Brief Description for the Use Case

The Solve Sudoku Puzzles use case allows the User to execute the Sudoku Solver program (Assignment6.exe) to process five input puzzle files (puzzle1.txt through puzzle5.txt), validate each initial 9x9 grid, apply a recursive backtracking algorithm to discover all valid Sudoku solutions, display progress to the console, and format/write all results sequentially into a single shared output file named output.txt

c. Flow of Events

- Basic Flow One or More Solutions Found:
- The User runs the executable from the command line.
- The System opens the single shared output.txt file for writing.

- The System initializes the file processing loop for $N = 1$ to 5 (puzzle1.txt to puzzle5.txt).
- The System initializes a blank 9x9 Grid filled with '_' characters.
- The System opens puzzleN.txt via the File System and reads 81 whitespace-separated characters ('1'-'9' or '_') into the 9x9 board.
- The System prints the puzzle filename and initial 9x9 grid to the console.
- The System initializes solutionCount = 0 and writes a distinct puzzle header section (e.g., identifying Puzzle N) to output.txt.
- The System validates that the initial board contains no duplicate digits in any row, column, or 3x3 subgrid.
- The System searches the 9x9 board row-by-row for the first empty cell ('_').
- If an empty cell is found at (r, c), the System filters out digits already present in row r, column c, and the corresponding 3x3 box to produce a list of legal candidate digits (1-9).
- For each legal candidate digit, the System places the digit at (r, c) and recursively invokes the solver; upon returning, it resets (r, c) back to '_' (backtracking).
- When a recursive call finds no empty cells remaining (base case), the System increments solutionCount, prints the solution to the console, and writes the solved 9x9 space-separated grid to output.txt under the current puzzle section.
- Once all recursive branches for puzzleN.txt are exhausted and solutionCount > 0, the System writes "Solutions found: n" (where n is solutionCount) to output.txt.
- The System repeats Steps 4-13 for the remaining puzzles until all 5 files have been processed, closes output.txt, and then terminates with exit code 0.

- Alternate Flow (No Solutions Found):
- Branch at Step 8 or Step 13: If the initial board is invalid (Step 8) OR all recursive candidate paths hit dead ends leaving solutionCount == 0 (Step 13), the System writes the single line "No solution found." to output.txt under the current puzzle section, prints "No solution found." to the console, and proceeds to the next puzzle.

- Exception Flow (Output File Cannot Be Opened):
- Branch at Step 2: If output.txt cannot be opened for writing due to file system permissions, the System outputs an error message (e.g., "Error opening output file: output.txt") and terminates execution.

d. Detailed Description for the Use Case Specification

-Use Case Name: Solve Sudoku Puzzles

-Primary Actor: User

-Secondary Actor: File System

-Preconditions:

-The compiled executable (Assignment6.exe) is present in the working environment.

-Input puzzle files (puzzle1.txt through puzzle5.txt) are formatted correctly and accessible via the File System.

-Postconditions:

-All valid solutions, counts, or "No solution found" notices for all five puzzles are successfully consolidated and written sequentially into the single output.txt file.

-Execution status, initial grids, and console logs are properly displayed.

-Trigger: The User executes the Sudoku Solver program from the command line interface.